บทที่ 9: การนำทางระหว่างหน้าจอ (Navigation Between Screens)

9.1 บทนำ: ก้าวข้ามหน้าจอเดียว

ลองนึกภาพอาคารขนาดใหญ่ที่ตกแต่งไว้สวยงาม มีห้องมากมาย เฟอร์นิเจอร์ครบทุกห้อง แต่พอเดินเข้าไปกลับพบว่าไม่มีประตู ไม่มีทางเดิน และไม่มีบันไดเชื่อมระหว่างห้องเลยแม้แต่แห่งเดียว คนที่เข้าไปจะติดอยู่ในห้องแรกตลอดไป แอปพลิเคชันก็เช่นกัน ต่อให้หน้าจอแรกออกแบบไว้สวยแค่ไหน ถ้าผู้ใช้แตะเพื่อไปยังหน้าอื่นไม่ได้ แอปนั้นก็ใช้งานไม่ได้ตั้งแต่ยังไม่เริ่ม การนำทางจึงเป็นเรื่องที่นักพัฒนามือใหม่มักมองข้าม ทั้งที่เป็นโครงสร้างที่ทุกหน้าจอต้องพึ่งพา

แอปพลิเคชันในโลกแห่งความเป็นจริงส่วนใหญ่ประกอบด้วยหน้าจอ (screens) หรือเพจ (pages) หลายหน้าที่ผู้ใช้ต้องนำทางไปมาระหว่างกัน ผู้ใช้อาจแตะรายการในลิสต์เพื่อดูรายละเอียด กดปุ่มเพื่อไปที่หน้าการตั้งค่า หรือกรอกฟอร์มเสร็จแล้วถูกพาไปยังหน้าจอยืนยัน การจัดการโฟลว์ระหว่างมุมมองต่างๆ นี้เป็นส่วนพื้นฐานของการพัฒนาแอป ใน Flutter การนำทาง (navigation) โดยทั่วไปจะจัดการโดยใช้ Routes และวิดเจ็ต Navigator

- Route หมายถึงหน้าจอหรือเพจภายในแอปพลิเคชันของคุณ
- Navigator จัดการกองซ้อน (stack) ของ routes เหล่านี้ เมื่อคุณนำทางไปยังหน้าจอใหม่ คุณ "push" route ของมันขึ้นไปบน stack เมื่อคุณย้อนกลับ คุณ "pop" route ปัจจุบันออกจาก stack เผยให้เห็นอันก่อนหน้า

บทนี้จะครอบคลุมแนวคิดและเทคนิคหลักสำหรับการสร้างการนำทางในแอปพลิเคชัน Flutter ของคุณ:

- ทำความเข้าใจวิดเจ็ต Navigator และแนวคิดเรื่อง route stack
- การใช้ MaterialPageRoute เพื่อกำหนดการเปลี่ยนผ่านระหว่างหน้าจอ
- การนำทางไปยังหน้าจอใหม่โดยใช้ Navigator.push()
- การกลับไปยังหน้าจอก่อนหน้าโดยใช้ Navigator.pop()
- การส่งข้อมูล *ไปยัง* หน้าจอใหม่เมื่อนำทาง
- การส่งคืนข้อมูล *จาก* หน้าจอเมื่อ pop กลับมา
- การจัดระเบียบการนำทางโดยใช้ Named Routes
- การจัดการ stack ที่มากกว่าการ push และ pop ทีละหน้า เช่นหน้าเข้าสู่ระบบและการออกจากระบบ
- การควบคุมการย้อนกลับด้วย PopScope เพื่อไม่ให้ข้อมูลที่ผู้ใช้กรอกไว้หาย
- การเลือกวิธีส่งข้อมูลให้เหมาะกับงาน และสัญญาณที่บอกว่าถึงเวลาต้องใช้การจัดการ state แบบอื่น

เมื่อจบบทนี้ คุณจะสามารถสร้างแอปพลิเคชันที่มีหลายหน้าจอ และสร้างการเปลี่ยนผ่านและการไหลของข้อมูลระหว่างหน้าจอเหล่านั้นได้อย่างราบรื่น

9.2 Navigator และ Route Stack

Flutter จัดการการนำทางโดยใช้วิดเจ็ต Navigator ซึ่งดูแล stack ของออบเจกต์ Route [1] คิดเหมือนกองจาน:

- หน้าจอที่ผู้ใช้มองเห็นอยู่ในปัจจุบันคือ route ที่อยู่ บนสุด ของ stack
- เมื่อคุณนำทางไปยังหน้าจอใหม่ คุณ push route ที่สอดคล้องกันขึ้นไปบนสุดของ stack หน้าจอใหม่จะปรากฏให้เห็น
- เมื่อคุณกดปุ่มย้อนกลับ (ไม่ว่าจะเป็นปุ่มย้อนกลับฮาร์ดแวร์บน Android หรือปุ่มย้อนกลับใน AppBar) Flutter จะ pop route บนสุดออกจาก stack เผยให้เห็นหน้าจอด้านล่าง

ถ้าเปรียบ route แต่ละตัวเป็นหน้าจอหนึ่งหน้า Navigator ก็คือศูนย์ควบคุมจราจรที่ตัดสินว่าหน้าไหนอยู่บนสุดและหน้าไหนรออยู่ข้างใต้ งานนี้มากกว่าการเปิดหน้า

ใหม่แล้วจบไป เพราะ stack คือความทรงจำของแอป Navigator ต้องจำว่าผู้ใช้เดินทางมาจากหน้าไหน กำลังจะไปไหน เมื่อกดย้อนกลับต้องกลับไปที่ใด และบางครั้งต้องพกข้อมูลอะไรติดตัวไปด้วย ซึ่งเป็นเรื่องของหัวข้อ 9.4 และ 9.5

MaterialApp (และ CupertinoApp) จะสร้าง Navigator ให้คุณโดยอัตโนมัติ และวางไว้ใน widget tree ใต้ตัวมันเอง [2] เมื่อเราเรียก Navigator.of(context).push(...) หรือรูปย่อ Navigator.push(context, ...) ซึ่งทำงานเหมือนกัน Flutter จะเริ่มจากตำแหน่งของ context ที่ส่งเข้าไป แล้วไล่ขึ้นไปตาม tree จนพบ Navigator ตัวที่ใกล้ที่สุด Scaffold ไม่ได้สร้าง Navigator และไม่เกี่ยวข้องกับการค้นหานี้ เงื่อนไขเดียวที่สำคัญคือ context ต้องอยู่ใต้ MaterialApp

กับดัก: context ที่มองไม่เห็น Navigator

ข้อผิดพลาดที่ผู้เริ่มต้นเจอบ่อยที่สุดในบทนี้เกิดจากการเขียนหน้าแรกรวมไว้ในเมธอด build ของ MyApp โดยตรง

```
// ตัวอย่างที่ผิด
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: ElevatedButton(
            // context ตัวนี้เป็นของ MyApp ซึ่งอยู่เหนือ MaterialApp ขึ้นไป
            // จึงมองไม่เห็น Navigator ที่ MaterialApp สร้างไว้ข้างล่าง
            onPressed: () => Navigator.push(
              context,
              MaterialPageRoute(
                builder: (context) => const SecondScreen(),
              ),
            ),
            child: const Text('ไปหน้าถัดไป'),
          ),
        ),
      ),
    );
  }
}
```

โค้ดนี้คอมไพล์ผ่าน แต่เมื่อกดปุ่มจะได้ข้อผิดพลาด Navigator operation requested with a context that does not include a Navigator. เพราะ context ที่ใช้เป็นของ MyApp ซึ่งอยู่เหนือ MaterialApp การไล่ขึ้นไปตาม tree จากจุดนั้นจึงไม่มีทางพบ Navigator ที่อยู่ข้างล่าง

ทางแก้คือแยกหน้าแรกออกเป็นวิดเจ็ตของตัวเอง เช่น HomeScreen แล้วใส่ไว้ใน home: const HomeScreen() เมธอด build ของ HomeScreen จะได้ context ของตัวเองซึ่งอยู่ใต้ MaterialApp นี่คือเหตุผลที่ตัวอย่างและปฏิบัติการในบทนี้แยกทุกหน้าจอเป็นคลาสของตัวเองเสมอ

9.3 การนำทางพื้นฐาน: Push และ Pop

การทำงานพื้นฐานสองอย่างสำหรับการนำทางคือการ push route ใหม่ขึ้นไปบน stack และการ pop route ปัจจุบันออกจาก stack

9.3.1 MaterialPageRoute

ก่อนที่เราจะ push ได้ เราต้องมีบางสิ่งที่จะ push นั่นก็คือ Route ซึ่งกำหนดทั้งหน้าจอที่จะแสดงและแอนิเมชันระหว่างเปลี่ยนหน้า Flutter มี MaterialPageRoute [3] ที่เลือกแอนิเมชันตามแพลตฟอร์มให้เอง บน iOS หน้าใหม่จะเลื่อนเข้ามาจากขอบขวา และผู้ใช้ปัดจากขอบซ้ายเพื่อย้อนกลับได้ บน Android จะใช้แอนิเมชันตามแบบของ Android รุ่นใหม่ ซึ่งเปลี่ยนไปตามเวอร์ชันของ Flutter (Flutter 3.44 ใช้แบบจางเข้าพร้อมเลื่อนเล็กน้อย และรองรับท่าย้อนกลับแบบ predictive back ของ Android 14 ขึ้นไป) MaterialPageRoute รับฟังก์ชัน builder เป็น argument ซึ่งฟังก์ชันนี้จะได้รับ BuildContext และต้องส่งคืนวิดเจ็ตของหน้าจอที่ต้องการนำทางไป

```
MaterialPageRoute(
  builder: (BuildContext context) {
    // ส่งคืนอินสแตนซ์ของวิดเจ็ตสำหรับหน้าจอใหม่
    return const SecondScreen();
  },
)
```

คำถามที่ผู้เรียนมักสงสัยคือ ทำไม Navigator.push จึงรับวิดเจ็ตของหน้าจอตรง ๆ ไม่ได้ ต้องมี MaterialPageRoute ห่อไว้อีกชั้น ลองเปรียบวิดเจ็ตของหน้าจอเป็นสิ่งของ ส่วน MaterialPageRoute คือซองหรือกล่องพัสดุที่ห่อสิ่งของนั้นไว้ บนกล่องมีคำสั่งกำกับว่าหน้าจอนี้ควรปรากฏขึ้นอย่างไรให้เข้ากับระบบปฏิบัติการที่กำลังรันอยู่ และเมื่อถูกนำออกจาก stack ควรหายไปอย่างไร Navigator จัดการกับกล่อง ไม่ได้จัดการกับสิ่งของข้างในโดยตรง

สิ่งที่ใส่ไว้ในกล่องก็ไม่ใช่หน้าจอที่สร้างเสร็จแล้ว แต่เป็นฟังก์ชัน builder ซึ่งเปรียบได้กับพิมพ์เขียว Navigator จะเก็บพิมพ์เขียวไว้ก่อน แล้วค่อยใช้สร้างหน้าจอจริงในจังหวะที่ route ถูกนำขึ้นไปบน stack เหตุผลหลักไม่ใช่เรื่องประหยัดหน่วยความจำ เพราะวิดเจ็ตเป็นเพียงออบเจกต์ขนาดเล็กที่บรรยายหน้าตา การสร้างล่วงหน้าหนึ่งตัวไม่ได้ทำให้เครื่องหน่วง เหตุผลที่แท้จริงคือหน้าจอต้องถูกสร้างภายใต้ตำแหน่งของ route นั้นใน widget tree และ route อาจต้องเรียก builder ซ้ำเมื่อมีการสร้างใหม่ การรับเป็นพิมพ์เขียวจึงทำให้ route เป็นผู้ควบคุมว่าจะสร้างเมื่อไรและที่ไหน

BuildContext ที่ builder ได้รับคือพิกัดนั้น เปรียบเหมือนพิกัด GPS ที่บอกว่าหน้าจอใหม่อยู่ตรงไหนของ widget tree เมื่อรู้พิกัดของตัวเอง หน้าจอจึงไล่ขึ้นไปหาสิ่งที่อยู่เหนือขึ้นไปได้ถูกต้อง เช่นธีม สี ฟอนต์ของแอป และ Navigator ตัวที่ดูแลมันอยู่ ถ้าไม่มีพิกัดนี้ หน้าจอจะเป็นเหมือนคนหลงทางที่ไม่รู้แม้แต่ว่าแอปใช้สีธีมอะไร และนี่คือเหตุผลเดียวกับกับดักในหัวข้อ 9.2 ที่ context ผิดตำแหน่งแล้วหา Navigator ไม่เจอ

9.3.2 Navigator.push()

ในการนำทาง *ไปยัง* หน้าจอใหม่ คุณ push route ของมันขึ้นไปบน stack ของ navigator โดยใช้ Navigator.push() [4]

```
// ภายใน onPressed callback เป็นต้น:
onPressed: () {
  Navigator.push( // Push route ใหม่ขึ้นไปบน stack
```

```
    context, // BuildContext ของวิดเจ็ตปัจจุบัน
    MaterialPageRoute(
      builder: (context) => const SecondScreen(), // สร้างหน้าจอปลายทาง
    ),
  );
},
```

เมื่อโค้ดนี้ทำงาน Flutter จะสร้าง SecondScreen, ทำแอนิเมชันให้มันปรากฏขึ้น (ตามการเปลี่ยนผ่านเริ่มต้นของ MaterialPageRoute), และเพิ่ม route ของมันไปไว้บนสุดของ navigator stack

9.3.3 Navigator.pop()

ในการย้อนกลับ (ปิดหน้าจอปัจจุบันและเผยหน้าจอก่อนหน้า) คุณ pop route ปัจจุบันออกจาก stack โดยใช้ Navigator.pop() [5] ซึ่งมักถูกกระตุ้นโดยลูกศรย้อนกลับใน AppBar (AppBar ใส่ปุ่มนี้ให้เองเมื่อพบว่ามีหน้าก่อนหน้าให้ย้อนกลับไป) ปุ่มหรือท่าย้อนกลับของระบบ หรือปุ่ม "Back" ที่สร้างขึ้นเอง

```
// ภายใน onPressed callback บน SecondScreen:
onPressed: () {
  Navigator.pop(context); // Pop route ปัจจุบัน (SecondScreen) ออกจาก stack
},
```

เมื่อ pop ถูกเรียก Flutter จะทำแอนิเมชันให้หน้าจอปัจจุบันหายไปและลบ route ของมันออกจาก stack ทำให้ route (หน้าจอ) ก่อนหน้ากลับมามองเห็นได้อีกครั้ง

เมื่อไม่มีหน้าให้ย้อนกลับ ถ้าเรียก Navigator.pop บนหน้าแรกซึ่งเป็น route เดียวใน stack Flutter จะนำ route นั้นออกโดยไม่แจ้งข้อผิดพลาดใด ๆ ผลคือหน้าจอว่างเปล่าสีดำ ถ้าปุ่มใดอาจถูกกดจากหน้าแรกได้ด้วย ให้ตรวจก่อนด้วย Navigator.canPop(context) หรือใช้ Navigator.maybePop(context) ซึ่งจะไม่ทำอะไรเลยถ้าไม่มีหน้าก่อนหน้า

9.4 การส่งข้อมูลไปยังหน้าจอใหม่

บ่อยครั้ง เมื่อนำทางไปยังหน้าจอใหม่ คุณต้องส่งข้อมูลบางอย่างไปด้วย (เช่น แตะสินค้าในลิสต์เพื่อนำทางไปยังหน้ารายละเอียดที่แสดงข้อมูลของสินค้า *เฉพาะนั้น*)

วิธีมาตรฐานในการส่งข้อมูล *ไปข้างหน้า* คือผ่าน constructor ของวิดเจ็ตที่แทนหน้าจอปลายทาง [6]

1. แก้ไขวิดเจ็ตปลายทาง: เพิ่ม properties ให้กับคลาสของวิดเจ็ตปลายทางเพื่อเก็บข้อมูลที่คุณต้องการรับ อัปเดต constructor ของมันให้ยอมรับข้อมูลนี้ (ปกติจะเป็น required named parameters)

```
// detail_screen.dart
class DetailScreen extends StatelessWidget {
```

```
  final String itemId; // Property สำหรับเก็บข้อมูล
  final String message;

  // Constructor ที่รับข้อมูล
  const DetailScreen({
    super.key,
    required this.itemId,
    required this.message,
  });

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('รายละเอียดไอเท็ม: $itemId')),
      body: Center(child: Text('ข้อความที่ได้รับ: $message')),
    );
  }
}
```

2. ส่งข้อมูลระหว่าง push: เมื่อสร้างอินสแตนซ์ของวิดเจ็ตปลายทางภายใน builder ของ MaterialPageRoute ให้ส่งข้อมูลเข้าไปใน constructor ของมัน

```
// ภายใน onPressed ของปุ่มในหน้าจอแรก
onPressed: () {
  String dataToSend = 'Item-123';
  String msg = 'สวัสดีจากหน้าโฮม!';
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => DetailScreen( // ส่งข้อมูลผ่าน constructor
        itemId: dataToSend,
        message: msg,
      ),
    ),
  );
},
```

ตอนนี้ DetailScreen สามารถเข้าถึง itemId และ message ผ่าน properties ของมันได้แล้ว (widget.itemId, widget.message หาก DetailScreen เป็น StatefulWidget)

ข้อดีของการส่งผ่าน constructor คือคอมไพเลอร์ตรวจให้ครบทุกจุด ถ้าลืมส่ง itemId หรือส่งตัวเลขแทนข้อความ โค้ดจะคอมไพล์ไม่ผ่านตั้งแต่ก่อนรัน ขอให้จำข้อดีนี้

ไว้ เพราะการส่งข้อมูลผ่าน named routes ในหัวข้อ 9.6 จะเสียข้อดีข้อนี้ไป

9.5 การส่งข้อมูลกลับจากหน้าจอ

บางครั้ง หน้าจอหนึ่งต้องการส่งผลลัพธ์กลับไปยังหน้าจอที่ push มันมา (เช่น หน้าจอการตั้งค่าส่งคืนค่าที่เลือก หรือหน้าจอการเลือกส่งคืนไอเท็มที่เลือก)

สิ่งนี้ทำได้โดยการให้ ผลลัพธ์ (result) เมื่อเรียก Navigator.pop() และใช้ await กับ Navigator.push() บนหน้าจอที่เรียก [7]

1. Pop พร้อมผลลัพธ์: บนหน้าจอที่ต้องส่งข้อมูลกลับ (เช่น SelectionScreen) ให้ส่งข้อมูลเป็น argument ที่สอง (ทางเลือก) ให้กับ Navigator.pop()

```
// ภายใน SelectionScreen, เช่น เมื่อเลือกไอเท็มแล้ว
ElevatedButton(
  onPressed: () {
    String selectedValue = 'ตัวเลือก ก';
    Navigator.pop(context, selectedValue); // ส่งผลลัพธ์กลับไป
  },
  child: const Text('เลือกตัวเลือก ก แล้วย้อนกลับ'),
)
```

2. Await ผลลัพธ์: บนหน้าจอที่ push SelectionScreen มา (HomeScreen) ให้ใช้ async และ await เมื่อเรียก Navigator.push() ค่าที่ส่งให้ pop() ใน SelectionScreen จะกลายเป็นค่าที่ได้จากการ push โค้ดด้านล่างเป็นเมธอดที่อยู่นอก build และต้องใช้ทั้ง context และ mounted จึงต้องวางไว้ในคลาส State ของ HomeScreen ที่เป็น StatefulWidget ถ้าวางไว้ใน StatelessWidget จะคอมไพล์ไม่ผ่าน เพราะเมธอดนอก build ไม่มีตัวแปร context ให้ใช้

```
// ภายในคลาส State ของ HomeScreen
Future<void> _navigateToSelectionScreen() async {
  // <String> บอกว่าหน้าที่เปิดจะส่ง String กลับมา result จึงมีชนิดเป็น String?
  final result = await Navigator.push<String>(
    context,
    MaterialPageRoute(builder: (context) => const SelectionScreen()),
  );

  // ระหว่างรอ หน้านี้อาจถูกปิดไปแล้ว ต้องตรวจก่อนใช้ context อีกครั้ง
  if (!mounted) return;

  // result เป็น null เมื่อผู้ใช้กดย้อนกลับโดยไม่ได้เลือก
  final text = result == null ? 'ไม่มีการเลือก' : 'เลือก: $result';
  ScaffoldMessenger.of(context)
    ..removeCurrentSnackBar()
    ..showSnackBar(SnackBar(content: Text(text)));
}

// ปุ่มในเมธอด build ของ HomeScreen
```

```
ElevatedButton(
  onPressed: _navigateToSelectionScreen, // เรียกเมธอด async
  child: const Text('ไปที่หน้าจอเลือก'),
)
```

เมื่อ SelectionScreen เรียก Navigator.pop(context, selectedValue) บรรทัด await Navigator.push(...) ใน HomeScreen จะทำงานต่อ และตัวแปร result จะได้รับค่า selectedValue

ถ้าลองคิดตามจะเห็นเรื่องที่ชวนสงสัย เมื่อ SelectionScreen เรียก pop หน้าจอนั้นกำลังจะถูกนำออกจาก stack และทำลายทิ้ง แล้วมันส่งข้อมูลกลับไปหาหน้าจอหลักที่รออยู่ได้อย่างไร คำตอบคือมันไม่ได้รอให้ตัวเองถูกทำลายก่อน ค่าที่แนบไปกับ Navigator.pop ถูกส่งมอบในจังหวะที่คำสั่ง pop ทำงานนั่นเอง

ปลายทางของค่านั้นคือ Navigator.push ซึ่งไม่ได้คืนค่าทันทีแบบคำสั่งทั่วไป แต่คืนค่าเป็น Future ซึ่งเป็นกลไกของภาษา Dart ที่สัญญาว่าจะมีข้อมูลกลับมาในอนาคต ภาพที่ช่วยให้เข้าใจคือการส่งหน่วยสอดแนมออกไปปฏิบัติภารกิจ บรรทัดที่มี await หน้า Navigator.push คือจุดที่ฟังก์ชันหยุดรอชั่วคราว ระหว่างนั้นหน้าจอใหม่หรือหน่วยสอดแนมก็ทำหน้าที่ของมันไป เมื่อผู้ใช้เลือกเสร็จและ pop พร้อมแนบข้อมูล รายงานก็เดินทางกลับมาที่บรรทัด await แล้วโค้ดบรรทัดถัดไปจึงทำงานต่อ เช่นนำผลลัพธ์ไปแสดงด้วย SnackBar

ระบุชนิดของผลลัพธ์ Navigator.push<String>(...) บอกว่าหน้าที่เปิดจะส่งค่า String กลับมา ผลของการ await จึงมีชนิดเป็น String? ถ้าไม่ระบุ result จะมีชนิดเป็น Object? และต้องตรวจชนิดเองด้วย result is String ทุกครั้ง

ทำไมผลลัพธ์จึงเป็น null ได้ ผู้ใช้ไม่จำเป็นต้องกดปุ่มเลือกของเราเสมอไป เขาอาจกดลูกศรย้อนกลับใน AppBar กดปุ่มย้อนกลับของ Android หรือปัดจากขอบจอบน iOS ทุกช่องทางนี้เรียก pop โดยไม่มีผลลัพธ์ เราจึงต้องรองรับกรณี null ทุกครั้ง

หน่วยสอดแนมจึงอาจกลับมามือเปล่าได้เสมอ ทุกครั้งที่ await ผลลัพธ์จากหน้าจออื่น ต้องมีป้อมยามตรวจก่อนนำไปใช้ต่อ ถ้า push ไม่ได้ระบุชนิด ผลลัพธ์จะเป็น dynamic และคอมไพเลอร์จะไม่เตือนเลยถ้าเรานำ null ไปคำนวณหรือเรียกเมธอดต่อ แอปจะพังตอนรัน ซึ่งโหมด debug แสดงเป็นจอสีแดงที่นักพัฒนาเรียกกันว่า Red Screen of Death การระบุ push<String> ทำให้ผลลัพธ์มีชนิด String? และ null safety ของ Dart จะบังคับให้เราเขียนป้อมยามนี้ตั้งแต่ตอนคอมไพล์

ตรวจ mounted หลัง await ระหว่างที่รอ หน้าหลักอาจถูกนำออกจาก stack ไปแล้ว เช่นมีโค้ดส่วนอื่นสั่งล้าง stack ด้วย pushAndRemoveUntil (หัวข้อ 9.7) ถ้ายังใช้ context ของหน้าที่ถูกนำออกไปแล้วจะเกิดข้อผิดพลาดตอนรัน บรรทัด if (!mounted) return; จึงเป็นด่านตรวจก่อนใช้ context อีกครั้ง และถ้าลืม ตัววิเคราะห์โค้ดของ Flutter จะเตือนด้วยกฎ use_build_context_synchronously [8] ในคลาส State ใช้ mounted ได้ทันที ส่วนในวิดเจ็ตแบบ StatelessWidget อย่างในปฏิบัติการท้ายบท ให้ใช้ context.mounted ซึ่งมีความหมายเดียวกัน

เรื่อง async และ await จะอธิบายเต็มรูปแบบในบทที่ 12 ในบทนี้ขอให้จำเพียงว่า await หน้า Navigator.push คือการหยุดรอจนกว่าหน้าที่เปิดไปจะถูก pop

9.6 Named Routes เพื่อการนำทางที่สะอาดขึ้น

การใช้ MaterialPageRoute โดยตรงภายใน Navigator.push ทำงานได้ดีสำหรับกรณีง่ายๆ อย่างไรก็ตาม เมื่อแอปของคุณเติบโตขึ้นพร้อมกับหน้าจอจำนวนมาก การส่งข้อมูลและการสร้าง routes แบบ inline อาจกลายเป็นเรื่องยุ่งเหยิงและซ้ำซ้อน

ลองนึกถึงแอปขายของออนไลน์ที่มีหน้าจอนับร้อย และมีจุดที่กดไปหน้า DetailScreen ได้จาก 50 ที่ต่างกัน ถ้าทุกจุดเขียน Navigator.push พร้อมสร้าง MaterialPageRoute และเรียก constructor เอง วันที่ต้องเพิ่มพารามิเตอร์ใน constructor ของ DetailScreen เราต้องไล่แก้ครบทั้ง 50 จุด ปัญหานี้เรียกว่าการ

ดูแลรักษาโค้ด (code maintainability) ที่แย่ลงเมื่อการนำทางกระจายอยู่ทั่วแอป

Named Routes แก้ปัญหานี้คล้ายป้ายบอกทางในห้างสรรพสินค้า แทนที่จะบอกทางกันปากต่อปาก เราตั้งชื่อให้ทุกเส้นทางเป็นสตริงคงที่ เช่นหน้าแรกคือ / หน้ารายละเอียดคือ /detail แล้วลงทะเบียนไว้ที่เดียวใน routes ของ MaterialApp จุดที่ต้องการนำทางเพียงบอกชื่อ Navigator จะเปิดดูในรายการเองว่าชื่อนั้นคู่กับหน้าจอใด

Named Routes มอบวิธีในการกำหนด navigation routes ทั้งหมดของคุณไว้ในตำแหน่งศูนย์กลาง (ปกติภายใน MaterialApp) และนำทางไปยัง routes เหล่านั้นโดยใช้ชื่อสตริงง่ายๆ [9]

ขั้นตอน:

1. กำหนดชื่อ Route: เป็นแนวปฏิบัติที่ดีที่จะกำหนดชื่อ route เป็น static constants ซึ่งมักจะอยู่ภายในคลาสของ screen widget นั้นๆ

```
// home_screen.dart
class HomeScreen extends StatelessWidget {
  static const routeName = '/'; // มักใช้ '/' สำหรับ initial route
  // ... เมธอด build ...
}

// detail_screen.dart
class DetailScreen extends StatelessWidget {
  static const routeName = '/detail';
  final String itemId;
  // ... constructor และ เมธอด build ...
}
```

2. กำหนดค่า MaterialApp: กำหนด initialRoute และ routes map ในวิดเจ็ต MaterialApp ของคุณ routes map จะเชื่อมโยงชื่อ route ที่เป็นสตริงกับฟังก์ชัน builder ที่สร้าง screen widgets ที่สอดคล้องกัน

```
// ใน main.dart -> MyApp.build()
MaterialApp(
  title: 'Named Routes Demo',
  // เริ่มแอปที่ route '/' (HomeScreen)
  initialRoute: HomeScreen.routeName,
  // กำหนด routes ที่มีอยู่และ builders ของมัน
  routes: {
    HomeScreen.routeName: (context) => const HomeScreen(),
    DetailScreen.routeName: (context) => const DetailScreen(itemId: 'Default ID',
message: 'From named route'), // ต้องการวิธีส่ง args! ดูด้านล่าง
    // เพิ่ม routes อื่นๆ ที่นี่:
    // SettingsScreen.routeName: (context) => const SettingsScreen(),
```

```
  },
)
```

สังเกตว่า DetailScreen ใน routes map ถูกสร้างด้วยค่าคงที่ 'Default ID' ทุกครั้ง routes map เป็นเพียงตารางที่จับคู่ชื่อกับวิธีสร้างหน้า ไม่มีช่องให้รับค่าที่ต่างกันในแต่ละครั้งที่ push นี่คือข้อจำกัดที่หัวข้อย่อยถัดไปจะแก้

3. นำทางโดยใช้ pushNamed: ใช้ Navigator.pushNamed() แทน Navigator.push(), โดยระบุ context และชื่อ route ที่เป็นสตริง

```
// นำทางไปยัง DetailScreen โดยใช้ชื่อ
onPressed: () {
  Navigator.pushNamed(context, DetailScreen.routeName);
},
```

การส่ง Arguments ด้วย Named Routes:

routes map พื้นฐานไม่รองรับการส่ง arguments ผ่าน constructors โดยตรงเหมือนที่เราทำก่อนหน้านี้ ในการจัดการปัญหานี้ คุณมีทางเลือกสองสามทาง:

- Navigator.pushNamed พร้อม arguments: คุณสามารถส่งออบเจกต์ arguments (ประเภทใดก็ได้ มักจะเป็น Map หรือคลาสที่คุณสร้างเอง) ผ่านพารามิเตอร์ arguments ของ pushNamed

```
// การ Push พร้อม arguments
Navigator.pushNamed(
  context,
  DetailScreen.routeName,
  arguments: {'itemId': 'Item-456', 'message': 'ข้อมูลผ่าน arguments'},
);
```

- การดึง Arguments: ในเมธอด build ของหน้าจอปลายทาง (หรือ didChangeDependencies ถ้าต้องการอ่านเพียงครั้งเดียวในวิดเจ็ตแบบ StatefulWidget) คุณดึง arguments เหล่านี้โดยใช้ ModalRoute.of(context)?.settings.arguments แล้ว cast ให้เป็นชนิดที่คาดหวัง ห้ามอ่านใน initState

```
// ในเมธอด build ของ DetailScreen
@override
Widget build(BuildContext context) {
  // ดึง arguments
  final args = ModalRoute.of(context)?.settings.arguments as Map<String, String>?;
  final itemId = args?['itemId'] ?? 'No ID'; // จัดการกรณี null ที่อาจเกิดขึ้น
```

```
  final message = args?['message'] ?? 'No Message';

  return Scaffold(
    appBar: AppBar(title: Text('Detail: $itemId')),
    body: Center(child: Text('Message: $message')),
  );
}
```

ทำไมอ่านใน initState ไม่ได้ ModalRoute.of(context) ไม่ได้แค่อ่านค่า แต่ลงทะเบียนให้วิดเจ็ตนี้ถูกสร้างใหม่เมื่อข้อมูลของ route เปลี่ยน การลงทะเบียนแบบนี้ทำใน initState ไม่ได้เพราะวิดเจ็ตยังติดตั้งเข้า tree ไม่เสร็จ Flutter จะหยุดด้วยข้อผิดพลาด dependOnInheritedWidgetOfExactType<_ModalScopeStatus>() ... was called before initState() completed ที่ที่ถูกต้องสำหรับการอ่านครั้งแรกคือ didChangeDependencies() ซึ่งทำงานต่อจาก initState ทันที บทที่ 11 จะใช้รูปแบบนี้ในปฏิบัติการ

การ cast เป็น Map เปราะบาง โค้ดด้านบนทำงานได้เพราะทุกค่าใน Map เป็น String แต่ถ้าวันหนึ่งหน้าต้นทางเพิ่ม 'count': 3 เข้าไปเพียงค่าเดียว Map ทั้งก้อนจะมีชนิดเป็น Map<String, Object> และการ cast จะพังตอนรันด้วยข้อความ type '_Map<String, Object>' is not a subtype of type 'Map<String, String>?' คอมไพเลอร์ไม่มีทางเตือนล่วงหน้า เพราะ arguments มีชนิดเป็น Object? ซึ่งรับได้ทุกอย่าง ชื่อคีย์ที่พิมพ์ผิดก็เช่นกัน args?['itemid'] จะได้ null แล้วตกไปใช้ค่าเริ่มต้นอย่างเงียบ ๆ

เปรียบ arguments ได้กับกล่องพัสดุที่ส่งไปพร้อมเส้นทางโดยไม่มีฉลากบอกว่าข้างในคืออะไร หน้าจอปลายทางต้องแกะกล่องเองแล้วเดาว่าข้างในเป็น Map, String หรือออบเจกต์สินค้า ลองนึกถึงผู้ใช้ที่แตะการแจ้งเตือนหรือลิงก์ที่เปิดเข้ามาในแอปโดยตรง (deep link) แล้วโค้ดส่วนนั้นส่ง arguments มาผิดรูปแบบหรือลืมส่ง หน้าจอปลายทางที่ cast ผิดชนิดจะพังทันทีต่อหน้าผู้ใช้

ทางแก้คือห่อข้อมูลทั้งหมดไว้ในคลาสของตัวเอง คอมไพเลอร์จะตรวจชื่อฟิลด์และชนิดให้ทั้งฝั่งส่งและฝั่งรับ เหลือจุดเสี่ยงเพียงบรรทัด cast บรรทัดเดียว [10]

```
// lib/screens/detail_args.dart
// ห่อข้อมูลทั้งหมดที่หน้ารายละเอียดต้องการไว้ในคลาสเดียว
class DetailArgs {
  final String itemId;
  final String message;

  const DetailArgs({required this.itemId, required this.message});
}

// หน้าต้นทาง
Navigator.pushNamed(
  context,
  DetailScreen.routeName,
  arguments: const DetailArgs(
    itemId: 'Item-456',
    message: 'ข้อมูลผ่าน arguments',
  ),
);
```

```dart
// หน้าปลายทาง ภายในเมธอด build
final args = ModalRoute.of(context)!.settings.arguments as DetailArgs;
```

- การใช้ onGenerateRoute: แนวทางที่ยืดหยุ่นกว่า (แต่ซับซ้อนกว่าเล็กน้อย) คือการใช้ property onGenerateRoute ใน MaterialApp แทน routes onGenerateRoute เป็นฟังก์ชันที่จะถูกเรียกหากมีการ push named route แต่ไม่พบใน routes map มันจะได้รับ RouteSettings (ซึ่งรวมถึงชื่อและ arguments) และต้องส่งคืน Route (เช่น MaterialPageRoute) สิ่งนี้ช่วยให้คุณสามารถแยกวิเคราะห์ arguments และสร้างวิดเจ็ตของคุณได้อย่างถูกต้องภายในคำจำกัดความของ route [11]

onGenerateRoute ทำหน้าที่เหมือนด่านตรวจคนเข้าเมือง ทุกชื่อที่ไม่อยู่ใน routes ต้องผ่านด่านนี้ ด่านจะดูว่าขอไปหน้าไหน ตรวจว่ากล่องพัสดุที่แนบมามีของถูกชนิดหรือไม่ ถ้าผิดหรือไม่มี ก็ส่งผู้ใช้ไปหน้าแจ้งข้อผิดพลาดแทนที่จะปล่อยให้แอปพัง ถ้าถูกต้อง ด่านจะแกะกล่องแล้วส่งข้อมูลเข้า constructor ของหน้าจอปลายทางให้เอง หน้าจอจึงกลับมารับข้อมูลแบบที่คอมไพเลอร์ตรวจได้เหมือนหัวข้อ 9.4 อย่างไรก็ตาม ด่านนี้ไม่ได้ทำให้ปลอดภัยสมบูรณ์ การตรวจชนิดของ arguments ยังเกิดขึ้นตอนรันหนึ่งจุดในตัวด่านเอง ต่างกันที่จุดนั้นมีเพียงจุดเดียวและเราเขียนทางออกไว้ให้แล้ว

ตัวอย่างด้านล่างย้าย DetailScreen ออกจาก routes map ไปสร้างใน onGenerateRoute แทน ข้อดีคือ DetailScreen กลับมารับข้อมูลผ่าน constructor แบบหัวข้อ 9.4 ได้ ตัวหน้าจอไม่ต้องรู้จัก ModalRoute เลย และการตรวจชนิดทั้งหมดรวมอยู่ในจุดเดียว

```dart
// ใน main.dart -> MyApp.build()
MaterialApp(
  initialRoute: HomeScreen.routeName,
  routes: {
    HomeScreen.routeName: (context) => const HomeScreen(),
    // ไม่ใส่ DetailScreen ไว้ที่นี่ เพราะต้องรับข้อมูลผ่าน constructor
  },
  // ถูกเรียกเมื่อชื่อที่ push มาไม่อยู่ใน routes
  onGenerateRoute: (settings) {
    final args = settings.arguments;
    final isDetail = settings.name == DetailScreen.routeName;
    if (isDetail && args is DetailArgs) {
      return MaterialPageRoute(
        settings: settings, // เก็บชื่อและ arguments ไว้กับ route ด้วย
        builder: (context) => DetailScreen(
          itemId: args.itemId,
          message: args.message,
        ),
      );
    }
    // ไม่รู้จักชื่อนี้ หรือข้อมูลไม่ครบ ส่งต่อให้ onUnknownRoute
    return null;
  },
  // ด่านสุดท้าย แทนที่แอปจะพังเพราะหา route ไม่เจอ
  onUnknownRoute: (settings) => MaterialPageRoute(
    builder: (context) => Scaffold(
```

```
      appBar: AppBar(title: const Text('ไม่พบหน้า')),
      body: Center(child: Text('ไม่มีหน้าชื่อ ${settings.name}')),
    ),
  ),
)
```

ลำดับการค้นหา เมื่อเรียก pushNamed Flutter จะหาชื่อใน routes ก่อน ถ้าไม่พบจึงเรียก onGenerateRoute ถ้า onGenerateRoute คืนค่า null จึงเรียก onUnknownRoute และถ้าไม่มี onUnknownRoute ด้วย แอปจะหยุดด้วยข้อผิดพลาด Could not find a generator for route ตัวอย่างนี้ใช้ args is DetailArgs ทั้งตรวจชนิดและแปลงชนิดในขั้นเดียว (Dart รู้ว่าภายในบล็อก if ตัวแปร args เป็น DetailArgs แน่นอน จึงเรียก args.itemId ได้ทันที) ถ้าผู้เรียกลืมส่ง arguments ผู้ใช้จะเห็นหน้าไม่พบหน้าแทนที่แอปจะพัง ส่วน settings: settings ทำให้ route ยังจำชื่อของตัวเองได้ ซึ่งจำเป็นเมื่อต้องการย้อนกลับไปหาหน้านี้ด้วยชื่อในหัวข้อ 9.7

Named routes ช่วยรวมรายชื่อหน้าจอทั้งหมดไว้ที่เดียว และเมื่อใช้ค่าคงที่อย่าง DetailScreen.routeName แทนการพิมพ์สตริงเอง ก็ลดโอกาสพิมพ์ชื่อผิดได้ แต่แลกกับการที่ arguments ไม่ผ่านการตรวจชนิดตอนคอมไพล์ ซึ่งต้องชดเชยด้วยคลาส arguments หรือ onGenerateRoute ตามที่แสดงไว้

ทิศทางปัจจุบันของ Flutter เอกสารทางการของ Flutter ไม่แนะนำ named routes สำหรับแอปส่วนใหญ่อีกต่อไป [1] เหตุผลหลักคือเรื่อง deep link เมื่อผู้ใช้เปิดลิงก์เข้ามายังหน้าใดหน้าหนึ่งของแอปโดยตรง named routes จะ push หน้านั้นเพิ่มขึ้นไปใหม่ทุกครั้งไม่ว่าผู้ใช้อยู่ที่หน้าใด และปรับพฤติกรรมนี้ไม่ได้ อีกทั้งบนเว็บยังไม่รองรับปุ่ม forward ของเบราว์เซอร์ แอปที่ต้องรองรับสองเรื่องนี้จะใช้แพ็กเกจจัดการเส้นทางอย่าง go_router [12] ซึ่งสร้างบน Router API ของ Flutter (มักเรียกกันว่า Navigator 2.0) อย่างไรก็ตาม push, pop และการส่งข้อมูลที่เรียนในบทนี้ยังเป็นพื้นฐานที่ใช้ต่อได้ทั้งหมด และ named routes ยังเพียงพอสำหรับแอปขนาดเล็กที่ไม่ต้องรองรับ deep link อย่างโปรเจกต์ในบทที่ 14 และ 15

9.7 จัดการ stack ให้มากกว่าการ push และ pop ทีละหน้า

ทุกตัวอย่างที่ผ่านมาเพิ่มหรือลดหน้าทีละหนึ่งหน้า แต่แอปจริงมีโฟลว์ที่ต้องการมากกว่านั้น ลองดูสามสถานการณ์ต่อไปนี้

- หน้าเข้าสู่ระบบ หลังเข้าสู่ระบบสำเร็จ ถ้าใช้ push ธรรมดา stack จะเป็น [Login, Home] ผู้ใช้กดย้อนกลับจากหน้าหลักแล้วกลับไปเจอหน้ากรอกรหัสผ่านอีก สิ่งที่ต้องการคือนำ Home มาแทนที่ Login ให้ stack เหลือ [Home]
- การออกจากระบบ ผู้ใช้อาจอยู่ลึกหลายชั้น เช่น [Home, Profile, Settings] เมื่อกดออกจากระบบต้องล้างทุกหน้าทิ้งให้เหลือ [Login] ไม่เช่นนั้นผู้ใช้คนถัดไปกดย้อนกลับแล้วจะเห็นข้อมูลของผู้ใช้คนเดิม
- กลับสู่หน้าแรก หลังสั่งซื้อสำเร็จจาก [Home, Product, Cart, Checkout] ต้องการกลับไปที่ Home รวดเดียว ไม่ใช่ให้ผู้ใช้กดย้อนกลับสามครั้ง

```
// หลังเข้าสู่ระบบสำเร็จ: เอาหน้าหลักมาวางแทนที่หน้า login
// กดย้อนกลับจากหน้าหลักจึงไม่วนกลับมาหน้า login อีก
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => const HomeScreen()),
);

// ออกจากระบบ: เปิดหน้า login แล้วล้างทุกหน้าที่อยู่ข้างใต้ทิ้ง
Navigator.pushAndRemoveUntil(
  context,
```

```
  MaterialPageRoute(builder: (context) => const LoginScreen()),
  (route) => false, // ตอบ false ทุกหน้า = ไม่เก็บหน้าไหนไว้เลย
);

// ย้อนกลับหลายชั้นรวดเดียวจนถึงหน้าแรกของ stack
Navigator.popUntil(context, (route) => route.isFirst);
```

Navigator.pushReplacement ทำงานเหมือน push แต่นำหน้าปัจจุบันออกไปพร้อมกัน Navigator.pushAndRemoveUntil push หน้าใหม่แล้วไล่นำหน้าข้างใต้ออกไปทีละหน้า จนกว่าฟังก์ชันเงื่อนไขจะตอบ true การตอบ false ทุกหน้าจึงหมายถึงล้างทั้ง stack ส่วน Navigator.popUntil นำหน้าบนสุดออกทีละหน้าจนกว่าเงื่อนไขจะเป็นจริง โดย route.isFirst คือหน้าล่างสุดของ stack [13]

ถ้าใช้ named routes จะมีเมธอดที่คู่กันคือ pushReplacementNamed และ pushNamedAndRemoveUntil และใช้เงื่อนไข ModalRoute.withName('/') เพื่อหยุดที่หน้าที่มีชื่อตรงกันได้

หน้าจอที่ถูกทับอยู่ใน stack ทำอะไรอยู่ สมมติว่าผู้ใช้กดลึกลงไปเรื่อย ๆ จนมีหน้าจอซ้อนกัน 20 ชั้น เขามองเห็นเพียงจานใบบนสุด แล้วอีก 19 ใบข้างใต้กำลังกินหน่วยความจำอยู่หรือไม่ คำตอบคือหน้าจอเหล่านั้นยังอยู่ใน widget tree ครบ พร้อม State และตัวแปรทุกตัว เพราะ MaterialPageRoute ตั้งค่า maintainState เป็น true ไว้ เมื่อผู้ใช้กดย้อนกลับจึงเห็นหน้าเดิมในสภาพเดิมทันที สิ่งที่ Flutter ช่วยประหยัดให้คือไม่จัดวางและไม่วาดหน้าที่ถูกทับ และหยุดแอนิเมชันของหน้าเหล่านั้นไว้ แต่งานที่เริ่มไปแล้ว เช่นการดึงข้อมูลจาก API หรือตัวจับเวลา ยังทำงานต่อไปและใช้หน่วยความจำตามเดิม

ข้อคิดสำหรับการออกแบบคือ อย่าปล่อยให้ stack ยาวขึ้นเรื่อย ๆ โดยไม่จำเป็น ใช้ pushReplacement เมื่อไม่ต้องการให้ย้อนกลับมาหน้าเดิม ใช้ popUntil หรือ pushAndRemoveUntil เมื่อจบโฟลว์หนึ่งแล้ว และหยุดงานเบื้องหลังใน dispose() ของหน้าจอเมื่อหน้านั้นถูกนำออกจาก stack

9.8 ควบคุมการย้อนกลับด้วย PopScope

บทที่ 8 สอนการสร้างฟอร์มรับข้อมูล ลองนึกถึงผู้ใช้ที่พิมพ์บันทึกยาวครึ่งหน้าแล้วเผลอปัดขอบจอ ทุกอย่างหายไปทันทีโดยไม่มีคำเตือน วิดเจ็ต PopScope [14] ให้เราดักการย้อนกลับไว้ได้ทุกช่องทาง ไม่ว่าจะมาจากลูกศรใน AppBar ปุ่มย้อนกลับของ Android หรือท่าปัดของ iOS

```
// lib/screens/edit_note_screen.dart
class EditNoteScreen extends StatefulWidget {
  const EditNoteScreen({super.key});

  @override
  State<EditNoteScreen> createState() => _EditNoteScreenState();
}

class _EditNoteScreenState extends State<EditNoteScreen> {
  bool _dirty = false; // ผู้ใช้พิมพ์อะไรไปแล้วหรือยัง

  Future<bool> _confirmDiscard() async {
    // showDialog ก็คือการ push route ใหม่ขึ้นบน stack เช่นกัน
    final discard = await showDialog<bool>(
      context: context,
```

```
      builder: (context) => AlertDialog(
        title: const Text('ทิ้งการแก้ไข?'),
        content: const Text('ข้อความที่พิมพ์ไว้จะหายไป'),
        actions: [
          TextButton(
            // ปิด dialog พร้อมส่งผลลัพธ์กลับ
            onPressed: () => Navigator.pop(context, false),
            child: const Text('แก้ต่อ'),
          ),
          TextButton(
            onPressed: () => Navigator.pop(context, true),
            child: const Text('ทิ้ง'),
          ),
        ],
      ),
    );
    // แตะนอกกรอบ dialog จะได้ null ถือว่าไม่ทิ้ง
    return discard ?? false;
  }

  @override
  Widget build(BuildContext context) {
    return PopScope(
      canPop: !_dirty, // ยังไม่ได้พิมพ์อะไร ปล่อยให้ย้อนกลับได้ตามปกติ
      onPopInvokedWithResult: (didPop, result) async {
        if (didPop) return; // ย้อนกลับไปเรียบร้อยแล้ว ไม่ต้องทำอะไรต่อ
        final discard = await _confirmDiscard();
        if (discard && context.mounted) {
          Navigator.pop(context); // ผู้ใช้ยืนยันแล้ว จึงสั่งปิดหน้าเอง
        }
      },
      child: Scaffold(
        appBar: AppBar(title: const Text('แก้ไขบันทึก')),
        body: TextField(
          onChanged: (_) {
            if (!_dirty) setState(() => _dirty = true);
          },
        ),
      ),
    );
  }
}
```

PopScope มีสองส่วนที่ทำงานคู่กัน canPop บอกไว้ล่วงหน้าว่าตอนนี้ยอมให้ย้อนกลับหรือไม่ ถ้าเป็น false การย้อนกลับจะถูกระงับ แล้ว Flutter จะเรียก onPopInvokedWithResult โดยส่ง didPop เป็น false เพื่อบอกว่ามีความพยายามย้อนกลับแต่ถูกกันไว้ เราจึงถามผู้ใช้ และสั่ง Navigator.pop เองเมื่อผู้ใช้ยืนยัน

เหตุที่ต้องบอกล่วงหน้าผ่าน canPop แทนที่จะตัดสินใจในจังหวะที่ผู้ใช้กด คือท่าปัดย้อนกลับของ iOS และ predictive back ของ Android ต้องรู้ตั้งแต่ก่อนนิ้วเริ่มลากว่าจะแสดงแอนิเมชันเผยหน้าก่อนหน้าหรือไม่ [15] ตำราหรือบทความรุ่นเก่าอาจใช้ WillPopScope ซึ่งเลิกใช้แล้วด้วยเหตุผลนี้ และพารามิเตอร์ onPopInvoked ของ PopScope รุ่นแรกก็ถูกแทนที่ด้วย onPopInvokedWithResult แล้วเช่นกัน

สังเกตด้วยว่า showDialog ก็คือการ push route ใหม่ขึ้นบน stack เดียวกัน ปุ่มใน dialog จึงปิดตัวเองด้วย Navigator.pop(context, false) และส่งผลลัพธ์กลับด้วยกลไกเดียวกับหัวข้อ 9.5 ทุกประการ

9.9 เลือกวิธีส่งข้อมูลให้เหมาะกับงาน

บทนี้แสดงวิธีส่งข้อมูลระหว่างหน้าจอไว้หลายวิธี แต่ละวิธีเหมาะกับงานต่างกัน

- ผ่าน constructor (9.4) เป็นค่าเริ่มต้นที่ควรเลือกก่อนเสมอ เพราะคอมไพเลอร์ตรวจให้ครบทั้งชื่อและชนิด
- ผ่าน arguments ของ named route (9.6) ใช้เมื่อโปรเจกต์ตัดสินใจใช้ named routes แล้ว ควรห่อด้วยคลาส arguments และถ้าเป็นไปได้ให้แปลงกลับเป็นพารามิเตอร์ของ constructor ใน onGenerateRoute
- ผ่านผลลัพธ์ของ pop (9.5) ใช้เมื่อหน้าที่เปิดไปมีหน้าที่ถามคำถามเดียวแล้วส่งคำตอบกลับ เช่นหน้าเลือกสี หน้าเลือกที่อยู่ หรือ dialog ยืนยัน
- ข้อมูลที่หลายหน้าใช้ร่วมกัน ไม่เหมาะกับวิธีใดในบทนี้เลย ถ้าหน้า A ต้องส่งข้อมูลผ่านหน้า B เพียงเพื่อให้ไปถึงหน้า C โดยที่ B ไม่ได้ใช้ข้อมูลนั้นเอง หรือหลายหน้าต้องเห็นค่าเดียวกันที่เปลี่ยนได้ตลอดเวลา นั่นคือสัญญาณว่าข้อมูลควรย้ายไปอยู่ในที่เก็บส่วนกลาง ซึ่งเป็นเรื่องของบทที่ 10

ข้อสังเกตอีกข้อหนึ่ง การส่งผ่าน constructor และ arguments เป็นการส่งค่าครั้งเดียว ณ เวลาที่ push หน้ารายการที่อยู่ข้างใต้ใน stack ไม่มีทางรู้ว่าหน้ารายละเอียดเปลี่ยนแปลงอะไรไป จนกว่าจะได้ผลลัพธ์กลับผ่าน pop แล้วนำไปอัปเดตหน้าจอของตัวเองด้วย setState

9.10 สรุปบทเรียนและสิ่งที่จะเรียนรู้ต่อไป

บทนี้ครอบคลุมพื้นฐานของการนำทางหน้าจอใน Flutter:

- Navigator จัดการ stack ของ Routes (หน้าจอ)
- MaterialPageRoute กำหนดการเปลี่ยนผ่านหน้าจอมาตรฐาน
- Navigator.push() เพิ่ม route เข้าไปใน stack (นำทางไปข้างหน้า)
- Navigator.pop() ลบ route บนสุดออก (นำทางย้อนกลับ)
- ข้อมูลถูกส่ง ไปข้างหน้า ยังหน้าจอใหม่ผ่าน widget constructors
- ข้อมูลถูกส่ง ย้อนกลับ โดยการให้ผลลัพธ์กับ pop() และใช้ await กับ push()
- Named Routes (MaterialApp.routes, Navigator.pushNamed) เสนอวิธีการจัดการการนำทางที่มีโครงสร้าง ซึ่งมักใช้กับ property arguments หรือ onGenerateRoute สำหรับการส่งข้อมูล
- Navigator.of(context) ค้นหา Navigator จากตำแหน่งของ context ขึ้นไปตาม tree ดังนั้น context ต้องอยู่ใต้ MaterialApp
- ระบุชนิดของผลลัพธ์ด้วย push<T> รองรับกรณี null เมื่อผู้ใช้ย้อนกลับเอง และตรวจ mounted ก่อนใช้ context หลัง await
- อ่าน arguments ใน build หรือ didChangeDependencies ห้ามอ่านใน initState และห่อ arguments ด้วยคลาสแทน Map
- pushReplacement, pushAndRemoveUntil และ popUntil จัดการ stack สำหรับโฟลว์เข้าสู่ระบบ ออกจากระบบ และกลับหน้าแรก
- PopScope ดักการย้อนกลับได้ทุกช่องทาง ใช้ป้องกันไม่ให้ข้อมูลในฟอร์มหาย

ตอนนี้คุณสามารถสร้างแอปพลิเคชันที่ประกอบด้วยหน้าจอหลายหน้าที่เชื่อมต่อกัน ช่วยให้ผู้ใช้สามารถย้ายไปมาระหว่างมุมมองและฟังก์ชันต่างๆ ได้

กิจกรรมปฏิบัติการสำหรับบทที่ 9:

แล็บจะเกี่ยวข้องกับการสร้างแอปหลายหน้าจออย่างง่ายเพื่อฝึกฝนเทคนิคการนำทาง:

1. สร้าง Screens: สร้าง StatelessWidget screens ง่ายๆ สองหน้าจอ: HomeScreen และ DetailScreen กำหนด Scaffold และ AppBar ให้แต่ละหน้าจอ
2. Basic Push/Pop: เพิ่ม ElevatedButton บน HomeScreen ใน onPressed ของมัน ใช้ Navigator.push และ MaterialPageRoute เพื่อนำทางไปยัง DetailScreen เพิ่ม ElevatedButton บน DetailScreen ที่ใช้ Navigator.pop เพื่อย้อนกลับ
3. Passing Data Forward: แก้ไข DetailScreen ให้ยอมรับ String (เช่น message) ใน constructor ของมันและแสดงใน body แก้ไขการเรียก push ใน HomeScreen เพื่อส่งข้อความสตริงเมื่อนำทางไปยัง DetailScreen
4. Returning Data Backward: เพิ่มปุ่มบน DetailScreen ที่เรียก Navigator.pop(context, 'ผลลัพธ์จาก Detail') แก้ไขการเรียก push ใน HomeScreen เพื่อใช้ async/await และจับผลลัพธ์ แสดงผลลัพธ์ใน HomeScreen โดยใช้ SnackBar
5. Refactor to Named Routes: กำหนด static routeName constants ในวิดเจ็ตทั้งสองหน้าจอ กำหนดค่า initialRoute และ routes map ใน MaterialApp เปลี่ยนการเรียกการนำทางเพื่อใช้ Navigator.pushNamed
6. (ทางเลือก) Passing Arguments with Named Routes: แก้ไขการเรียก pushNamed เพื่อส่ง arguments แก้ไขเมธอด build ของ DetailScreen เพื่อดึงและแสดง arguments เหล่านี้โดยใช้ ModalRoute.of(context)?.settings.arguments

บทต่อไป:

แม้ว่า setState จะดีสำหรับ state เฉพาะที่ การจัดการ state ข้ามหลายหน้าจอหรือ widget trees ที่ซับซ้อนจะกลายเป็นเรื่องยาก บทที่ 10: ความรู้เบื้องต้นเกี่ยวกับแพ็กเกจการจัดการ State (Provider) จะแนะนำแนวทางการจัดการ state ที่ปรับขนาดได้และเป็นระเบียบมากขึ้นโดยใช้แพ็กเกจ Provider ที่ได้รับความนิยม

9.11 คำถามชวนคิดท้ายบท

สมมติว่าเรากำลังสร้างแอปร้านค้าที่มีสามหน้า คือหน้ารายการสินค้า หน้ารายละเอียดสินค้า และหน้าตะกร้า ปุ่มเพิ่มลงตะกร้าอยู่ในหน้ารายละเอียด ส่วนตัวเลขจำนวนสินค้าในตะกร้าต้องแสดงที่มุมขวาบนของทั้งสามหน้า

ด้วยเครื่องมือจากบทนี้เพียงอย่างเดียว คือ constructor, arguments และผลลัพธ์จาก pop ลองออกแบบดูว่ารายการในตะกร้าควรเก็บไว้ที่หน้าใด เมื่อผู้ใช้เพิ่มสินค้าจากหน้ารายละเอียดแล้วกดย้อนกลับ หน้ารายการจะรู้ได้อย่างไรว่าต้องเปลี่ยนตัวเลข ถ้าผู้ใช้เปิดหน้าตะกร้าจากหน้ารายละเอียดโดยตรง ข้อมูลต้องเดินทางผ่านกี่ทอด และถ้าในอนาคตมีหน้าที่สี่และห้าที่ต้องแสดงตัวเลขเดียวกัน ต้องแก้โค้ดกี่จุด

ลองร่างคำตอบด้วยตัวเองก่อน แล้วนำไปเทียบกับแนวทางในบทที่ 10 ซึ่งเปิดบทด้วยสถานการณ์ตะกร้าสินค้าแบบเดียวกันนี้

เอกสารอ้างอิง (References)

[1] Flutter Team, "Navigation and routing," *Flutter Documentation*. Accessed: Apr. 29, 2025. [Online]. Available: https://docs.flutter.dev/ui/navigation

[2] Flutter Team, "Navigator class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/Navigator-class.html

[3] Flutter Team, "MaterialPageRoute class," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/material/MaterialPageRoute-class.html

[4] Flutter Team, "Navigator class - push method," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/Navigator/push.html

[5] Flutter Team, "Navigator class - pop method," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/widgets/Navigator/pop.html

[6] Flutter Team, "Send data to a new screen," *Flutter Documentation*. Accessed: Apr. 29, 2025. [Online]. Available: https://docs.flutter.dev/cookbook/navigation/passing-data

[7] Flutter Team, "Return data from a screen," *Flutter Documentation*. Accessed: Apr. 29, 2025. [Online]. Available: https://docs.flutter.dev/cookbook/navigation/returning-data

[8] Dart Team, "use_build_context_synchronously," *Dart Linter Rules*. Accessed: Sep. 24, 2026. [Online]. Available: https://dart.dev/tools/linter-rules/use_build_context_synchronously

[9] Flutter Team, "Navigate with named routes," *Flutter Documentation*. Accessed: Apr. 29, 2025. [Online]. Available: https://docs.flutter.dev/cookbook/navigation/named-routes

[10] Flutter Team, "Pass arguments to a named route," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/cookbook/navigation/navigate-with-arguments

[11] Flutter Team, "MaterialApp class - onGenerateRoute property," *Flutter API Docs*. Accessed: Apr. 29, 2025. [Online]. Available: https://api.flutter.dev/flutter/material/MaterialApp/onGenerateRoute.html

[12] Flutter Team, "go_router," *pub.dev*. Accessed: Sep. 24, 2026. [Online]. Available: https://pub.dev/packages/go_router

[13] Flutter Team, "Navigator class - pushAndRemoveUntil method," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/widgets/Navigator/pushAndRemoveUntil.html

[14] Flutter Team, "PopScope class," *Flutter API Docs*. Accessed: Sep. 24, 2026. [Online]. Available: https://api.flutter.dev/flutter/widgets/PopScope-class.html

[15] Flutter Team, "Add support for Android's predictive back gesture," *Flutter Documentation*. Accessed: Sep. 24, 2026. [Online]. Available: https://docs.flutter.dev/platform-integration/android/predictive-back

ปฏิบัติการบทที่ 9: การนำทางระหว่างหน้าจอ

วัตถุประสงค์: เพื่อฝึกฝนการสร้างการนำทางระหว่างหลายหน้าจอโดยใช้ Navigator.push, Navigator.pop, การส่งข้อมูลไปข้างหน้าและย้อนกลับ และการ refactor เพื่อใช้ named routes

เวลาโดยประมาณ: 2 ชั่วโมง

สิ่งที่ต้องมี: เสร็จสิ้นปฏิบัติการบทที่ 8 (คุ้นเคยกับ StatefulWidget, องค์ประกอบ UI พื้นฐาน), ติดตั้ง VS Code (หรือ Android Studio), Emulator/อุปกรณ์จริงกำลังทำงาน

การตั้งค่า: สร้างไฟล์ Screen

1. คำแนะนำ: สร้าง โปรเจคชื่อ ‘navigate_route’ แล้วให้ไนโฟลเดอร์ lib ของโปรเจกต์ของคุณ สร้างโฟลเดอร์ใหม่ชื่อ screens
2. คำแนะนำ: ภายในโฟลเดอร์ screens สร้างไฟล์ใหม่สองไฟล์:
    - home_screen.dart
    - detail_screen.dart

- ผลลัพธ์ที่คาดหวัง (โครงสร้างโปรเจกต์):

```
lib/
├── screens/
│   ├── home_screen.dart
│   └── detail_screen.dart
└── main.dart
```

3. คำแนะนำ: เนื้อหาพื้นฐานสำหรับ home_screen.dart สร้าง StatelessWidget ชื่อ HomeScreen กำหนด Scaffold พร้อม AppBar ชื่อ "หน้าหลัก" และ ElevatedButton ที่จัดกึ่งกลางใน body ชื่อ "ไปที่รายละเอียด" onPressed ของปุ่มสามารถเว้นว่างไว้ก่อน () {}

```dart
// lib/screens/home_screen.dart
import 'package:flutter/material.dart';

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('หน้าหลัก')),
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            // ตรรกะการนำทางจะมาที่นี่
            debugPrint('กดปุ่ม ไปที่รายละเอียด');
          },
          child: const Text('ไปที่รายละเอียด'),
        ),
      ),
    );
  }
}
```

4. คำแนะนำ: เนื้อหาพื้นฐานสำหรับ detail_screen.dart สร้าง StatelessWidget ชื่อ DetailScreen กำหนด Scaffold พร้อม AppBar ชื่อ "หน้ารายละเอียด" และวิดเจ็ต Text ที่จัดกึ่งกลางใน body ว่า "รายละเอียดอยู่ที่นี่" เพิ่ม ElevatedButton ใต้ข้อความชื่อ "ย้อนกลับ" พร้อม onPressed ที่ว่างเปล่า

```dart
// lib/screens/detail_screen.dart
```

```dart
import 'package:flutter/material.dart';

class DetailScreen extends StatelessWidget {
  const DetailScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('หน้ารายละเอียด')),
      body: Center(
        child: Column( // ใช้ Column เพื่อซ้อน Text และ Button
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            const Text('รายละเอียดอยู่ที่นี่'),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                // ตรรกะการย้อนกลับจะมาที่นี่
                debugPrint('กดปุ่ม ย้อนกลับ');
              },
              child: const Text('ย้อนกลับ'),
            ),
          ],
        ),
      ),
    );
  }
}
```

5. คำแนะนำ: อัปเดต main.dart แก้ไข main.dart เพื่อ import home_screen.dart และตั้งค่า HomeScreen เป็นวิดเจ็ต home ใน MaterialApp ลบโค้ดแล็บก่อนหน้าออกจาก MyHomePage หรือที่คล้ายกันหากมีอยู่

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'screens/home_screen.dart'; // Import หน้าจอหลัก

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});
```

```
  @override
  Widget build(BuildContext context) {
    return const MaterialApp(
      debugShowCheckedModeBanner: false,
      home: HomeScreen(), // เริ่มต้นด้วย HomeScreen
    );
  }
}
```

6. คำแนะนำ: รันแอป
    - ผลลัพธ์ที่คาดหวัง (UI): แอปเริ่มต้นที่ "หน้าหลัก" แสดงปุ่ม "ไปที่รายละเอียด" การแตะปุ่มตอนนี้ยังไม่ทำอะไรนอกจากพิมพ์ข้อความไปยัง console (ใช้ debugPrint แทน print เพราะ print จะถูกตัววิเคราะห์โค้ดเตือนด้วยกฎ avoid_print)

แบบฝึกหัดที่ 1: การนำทาง Push/Pop พื้นฐาน

1. คำแนะนำ: สร้าง push ใน HomeScreen ใน home_screen.dart ค้นหา callback onPressed สำหรับปุ่ม "ไปที่รายละเอียด" แทนที่คำสั่ง print ใช้ Navigator.push() เพื่อนำทางไปยัง DetailScreen คุณจะต้อง import detail_screen.dart

```
// lib/screens/home_screen.dart
import 'package:flutter/material.dart';
import 'detail_screen.dart'; // Import DetailScreen

class HomeScreen extends StatelessWidget {
  // ... constructor ...
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // ... AppBar ...
      body: Center(
        child: ElevatedButton(
          onPressed: () {
            Navigator.push( // นำทางไปยัง DetailScreen
              context,
              MaterialPageRoute(builder: (context) => const DetailScreen()),
            );
          },
          child: const Text('ไปที่รายละเอียด'),
        ),
      ),
    );
  }
}
```

2. คำแนะนำ: สร้าง pop ใน DetailScreen ใน detail_screen.dart ค้นหา callback onPressed สำหรับปุ่ม "ย้อนกลับ" แทนที่คำสั่ง print ใช้ Navigator.pop(context) เพื่อกลับไปยังหน้าจอก่อนหน้า

```dart
// lib/screens/detail_screen.dart
// ... imports ...
class DetailScreen extends StatelessWidget {
  // ... constructor ...
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      // ... AppBar ...
      body: Center(
        child: Column(
          // ... Text ...
          children: <Widget>[
            const Text('รายละเอียดอยู่ที่นี่'),
            const SizedBox(height: 20),
            ElevatedButton(
              onPressed: () {
                Navigator.pop(context); // กลับไปยังหน้าจอก่อนหน้า
              },
              child: const Text('ย้อนกลับ'),
            ),
          ],
        ),
      ),
    );
  }
}
```

3. คำแนะนำ: รันแอป (อาจต้อง Hot Restart)
   - ผลลัพธ์ที่คาดหวัง (พฤติกรรม): การแตะ "ไปที่รายละเอียด" บนหน้าหลักตอนนี้นำทางไปยังหน้ารายละเอียด การแตะ "ย้อนกลับ" บนหน้ารายละเอียดนำทางกลับไปยังหน้าหลัก ปุ่มย้อนกลับของอุปกรณ์ (ฮาร์ดแวร์ Android หรือท่าทางปัด iOS) ควรทำงานเหมือนกับการ pop จากหน้ารายละเอียดด้วย

แบบฝึกหัดที่ 2: การส่งข้อมูลไปข้างหน้า

1. คำแนะนำ: แก้ไข DetailScreen ให้รับข้อมูล ใน detail_screen.dart:

- เพิ่ม property final String message; ให้กับคลาส DetailScreen
- อัปเดต constructor ให้ต้องการ message นี้: const DetailScreen({super.key, required this.message});
- เปลี่ยนวิดเจ็ต Text ใน body ให้แสดงข้อความที่ได้รับ: Text(message)

```dart
// lib/screens/detail_screen.dart
import 'package:flutter/material.dart';

class DetailScreen extends StatelessWidget {
  final String message; // Property สำหรับเก็บข้อมูล

  // Constructor ที่อัปเดตแล้ว
  const DetailScreen({super.key, required this.message});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('หน้ารายละเอียด')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: <Widget>[
            Text(message), // แสดงข้อความที่ส่งมา
            const SizedBox(height: 20),
            ElevatedButton( /* ... ปุ่มย้อนกลับ ... */ ),
          ],
        ),
      ),
    );
  }
}
```

2. คำแนะนำ: ส่งข้อมูลจาก HomeScreen ใน home_screen.dart แก้ไขการเรียก Navigator.push เมื่อสร้างอินสแตนซ์ DetailScreen ภายใน MaterialPageRoute ให้ส่งค่าสำหรับ argument message

```dart
// lib/screens/home_screen.dart
// ... imports ...
onPressed: () {
  Navigator.push(
    context,
    MaterialPageRoute(
      builder: (context) => const DetailScreen( // ส่งข้อมูล
        message: 'ข้อมูลส่งมาจากหน้าหลัก!',
      ),
```

```
    ),
  );
},
```

3. คำแนะนำ: รันแอป (อาจต้อง Hot Restart)
    - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): นำทางจากหน้าหลักไปยังหน้ารายละเอียด หน้ารายละเอียดควรแสดงข้อความ "ข้อมูลส่งมาจากหน้าหลัก!" แทน "รายละเอียดอยู่ที่นี่"

แบบฝึกหัดที่ 3: การส่งข้อมูลย้อนกลับ

1. คำแนะนำ: แก้ไข DetailScreen ให้ pop พร้อมข้อมูล ใน detail_screen.dart เปลี่ยน onPressed ของปุ่ม "ย้อนกลับ" ให้รวมผลลัพธ์สตริงเมื่อเรียก pop

```
// ภายใน onPressed ของปุ่ม "ย้อนกลับ" ของ DetailScreen
onPressed: () {
  Navigator.pop(context, 'สำเร็จ! ข้อมูลจากรายละเอียด'); // ส่งสตริงนี้กลับ
},
```

2. คำแนะนำ: แก้ไข HomeScreen ให้ await และจัดการผลลัพธ์
    - เปลี่ยน callback onPressed ของปุ่ม "ไปที่รายละเอียด" ใน HomeScreen ให้เป็น async
    - ใช้ await เมื่อเรียก Navigator.push เก็บผลลัพธ์ไว้ในตัวแปร
    - ตรวจสอบว่าผลลัพธ์ไม่ใช่ null และแสดงมันโดยใช้ ScaffoldMessenger.of(context).showSnackBar(...)

```
// lib/screens/home_screen.dart
// ... imports ...
onPressed: () async { // ทำให้ onPressed เป็น async
  final result = await Navigator.push( // await ผลลัพธ์
    context,
    MaterialPageRoute(
      builder: (context) => const DetailScreen(
        message: 'ข้อมูลส่งมาจากหน้าหลัก!',
      ),
    ),
  );

  // จัดการผลลัพธ์เมื่อ DetailScreen pop กลับมา
  if (context.mounted) { // ตรวจสอบว่า widget ยังอยู่ใน tree
    String messageToShow;
```

```
    if (result != null && result is String) {
      messageToShow = 'ผลลัพธ์: $result';
    } else {
      messageToShow = 'ไม่มีผลลัพธ์ส่งกลับมา';
    }
    ScaffoldMessenger.of(context)
      ..removeCurrentSnackBar()
      ..showSnackBar(SnackBar(content: Text(messageToShow)));
  }
},
```

- *หมายเหตุ: ต้องตรวจ if (context.mounted) ทุกครั้งก่อนใช้ context หลัง await เหตุผลอธิบายไว้ในหัวข้อ 9.5 HomeScreen ในปฏิบัติการเป็น StatelessWidget จึงใช้ context.mounted แทน mounted ของคลาส State*

3. คำแนะนำ: รันแอป (ต้อง Hot Restart)
   - ไปที่หน้ารายละเอียด
   - แตะปุ่ม "ย้อนกลับ"
   - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): เมื่อคุณกลับมาที่หน้าหลักหลังจากแตะ "ย้อนกลับ" SnackBar ควรปรากฏขึ้นที่ด้านล่างแสดง "ผลลัพธ์: สำเร็จ! ข้อมูลจากรายละเอียด" หากคุณใช้ปุ่มย้อนกลับของระบบ/ท่าทางปัด ผลลัพธ์จะเป็น null และ SnackBar ควรแสดง "ไม่มีผลลัพธ์ส่งกลับมา"

แบบฝึกหัดที่ 4: Refactor เป็น Named Routes

1. คำแนะนำ: กำหนดชื่อ route
   - ใน home_screen.dart เพิ่ม: static const routeName = '/'; ภายในคลาส HomeScreen
   - ใน detail_screen.dart เพิ่ม: static const routeName = '/detail'; ภายในคลาส DetailScreen (ลบ property message และ argument ของ constructor ออกไปก่อน เราจะจัดการ arguments ด้วยวิธีอื่น) รีเซ็ต Text ใน body เป็นข้อความง่ายๆ เช่น 'เนื้อหาหน้ารายละเอียด'

```
// lib/screens/detail_screen.dart
class DetailScreen extends StatelessWidget {
  static const routeName = '/detail'; // กำหนดชื่อ route

  // ลบ property message และอัปเดต constructor
  const DetailScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('หน้ารายละเอียด (Named)')),
      body: Center( /* ... Column พร้อม Text('เนื้อหาหน้ารายละเอียด') และปุ่มย้อนกลับ ...
```

```
*/ ),
    );
  }
}
```

2. คำแนะนำ: กำหนดค่า MaterialApp ใน main.dart แก้ไข MaterialApp:
   - ตั้งค่า initialRoute: HomeScreen.routeName;
   - ลบ property home ออก
   - เพิ่ม property routes สร้าง map ที่เชื่อมโยงชื่อ route กับฟังก์ชัน builder Import ไฟล์ screen ทั้งสองไฟล์

```dart
// lib/main.dart
import 'package:flutter/material.dart';
import 'screens/home_screen.dart';
import 'screens/detail_screen.dart';

// ... main() ...

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp( // กำหนดค่า routes
      debugShowCheckedModeBanner: false,
      initialRoute: HomeScreen.routeName, // ชื่อ route เริ่มต้น
      routes: {
        HomeScreen.routeName: (context) => const HomeScreen(),
        DetailScreen.routeName: (context) => const DetailScreen(),
        // เพิ่ม screens อื่นๆ ที่นี่ภายหลัง
      },
    );
  }
}
```

3. คำแนะนำ: ใช้ pushNamed ใน HomeScreen แก้ไข onPressed ของปุ่ม "ไปที่รายละเอียด" ใน HomeScreen แทนที่ Navigator.push ด้วย Navigator.pushNamed โดยส่ง DetailScreen.routeName ลบ async/await และการจัดการผลลัพธ์ออกไปก่อน

```dart
// lib/screens/home_screen.dart -> onPressed
onPressed: () {
```

```
  Navigator.pushNamed(context, DetailScreen.routeName); // ใช้ pushNamed
},
```

4. คำแนะนำ: รันแอป (ต้องรีสตาร์ทเต็มรูปแบบหลังเปลี่ยน MaterialApp)
    - ผลลัพธ์ที่คาดหวัง (พฤติกรรม): การนำทางควรทำงานเหมือนกับในแบบฝึกหัดที่ 1 (push ไปยัง DetailScreen, pop กลับมา) แต่ตอนนี้ใช้ named routes

แบบฝึกหัดที่ 5: การส่ง Arguments ด้วย Named Routes

1. คำแนะนำ: แก้ไข pushNamed เพื่อส่ง arguments ใน HomeScreen เปลี่ยน pushNamed ให้รวม arguments map

```
// lib/screens/home_screen.dart -> onPressed
onPressed: () {
  Navigator.pushNamed(
    context,
    DetailScreen.routeName,
    arguments: { // ส่ง arguments เป็น Map
      'id': 'ARG-789',
      'title': 'ข้อมูลจาก Named Route Args',
    },
  );
},
```

2. คำแนะนำ: ดึง arguments ใน DetailScreen ใน detail_screen.dart แก้ไขเมธอด build:
    - ใช้ ModalRoute.of(context)?.settings.arguments เพื่อรับ arguments map
    - Cast arguments ให้เป็นประเภทที่คาดหวัง (Map<String, String>?)
    - ดึงค่า 'id' และ 'title' (จัดการกรณี null ที่อาจเกิดขึ้นโดยใช้ ??)
    - แสดงค่าเหล่านี้ใน title ของ AppBar และ Text ใน body
    - คงปุ่ม "ย้อนกลับ" ที่ใช้ Navigator.pop(context); ไว้

```
// lib/screens/detail_screen.dart -> เมธอด build
@override
Widget build(BuildContext context) {
  // ดึง arguments
  final args = ModalRoute.of(context)?.settings.arguments as Map<String, String>?;

  // ใช้ arguments, กำหนดค่าเริ่มต้นหากเป็น null
```

```
  final String itemId = args?['id'] ?? 'ไม่มี ID';
  final String pageTitle = args?['title'] ?? 'รายละเอียด';

  return Scaffold(
    appBar: AppBar(title: Text('ไอเท็ม: $itemId')), // ใช้ ID ที่ดึงมา
    body: Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: <Widget>[
          Text('ได้รับ: $pageTitle'), // ใช้ title ที่ดึงมา
          const SizedBox(height: 20),
          ElevatedButton(
            onPressed: () {
              Navigator.pop(context); // Pop กลับง่ายๆ
            },
            child: const Text('ย้อนกลับ'),
          ),
        ],
      ),
    ),
  );
}
```

3. คำแนะนำ: รันแอป (อาจต้องรีสตาร์ทเต็มรูปแบบ)
    - ผลลัพธ์ที่คาดหวัง (UI & พฤติกรรม): นำทางจากหน้าหลักไปยังหน้ารายละเอียด AppBar ของหน้ารายละเอียดควรแสดง "ไอเท็ม: ARG-789" และข้อความใน body ควรแสดง "ได้รับ: ข้อมูลจาก Named Route Args" ซึ่งแสดงให้เห็นว่าข้อมูลถูกส่งผ่านสำเร็จโดยใช้ named route arguments

ความท้าทายเพิ่มเติม

ข้อที่ 1 เปลี่ยน arguments ในแบบฝึกหัดที่ 5 จาก Map เป็นคลาส DetailArgs แล้วย้ายการสร้าง DetailScreen ไปไว้ใน onGenerateRoute ตามหัวข้อ 9.6 ให้ DetailScreen รับค่าผ่าน constructor จากนั้นลองเรียก pushNamed โดยไม่ส่ง arguments และเพิ่ม onUnknownRoute ให้แอปแสดงหน้าไม่พบหน้าแทนการพัง

ข้อที่ 2 เพิ่ม TextField ในหน้ารายละเอียด แล้วครอบหน้าด้วย PopScope ตามหัวข้อ 9.8 เมื่อผู้ใช้พิมพ์แล้วพยายามย้อนกลับไม่ว่าช่องทางใด ให้แสดง dialog ถามก่อนทิ้งข้อความ ทดสอบทั้งปุ่มย้อนกลับในหน้า ลูกศรใน AppBar และปุ่มหรือท่าย้อนกลับของระบบ

ข้อที่ 3 เพิ่ม LoginScreen เป็นหน้าแรก เมื่อกดเข้าสู่ระบบให้ไปหน้าหลักด้วย pushReplacement และเพิ่มปุ่มออกจากระบบในหน้ารายละเอียดที่ใช้ pushAndRemoveUntil กลับไปหน้า LoginScreen แล้วตรวจว่ากดย้อนกลับจากทุกหน้าแล้วไม่วนกลับไปหน้าที่ควรถูกล้างไปแล้ว

ยินดีด้วย! ตอนนี้คุณได้สร้างรูปแบบการนำทางต่างๆ ใน Flutter สำเร็จแล้ว รวมถึง push/pop พื้นฐาน, การส่งข้อมูลไปข้างหน้าผ่าน constructors, การส่งข้อมูลย้อนกลับโดยใช้ await และผลลัพธ์ pop, และการจัดระเบียบการนำทางด้วย named routes และ arguments